LMS / Meshmerize — Fresher Selection Tasks

Submit all nine tasks as a single GitHub repository. Justify every decision. Related work may be combined.

All implementation tasks target **ESP32 (Arduino framework, C++)**. Hardware references use the spec sheet in Appendix A.

Tasks are ordered easiest to hardest. Attempt them in sequence.

Point Distribution

Task	Title	Points
1	Basic Coding	15
2	Datasheet Literacy	18
3	PID Control	24
4	BOM Preparation	27
5	Power Budget & Battery Selection	30
6	Electronics & Hardware	36
7	Wireless Sensor Diagnostics	39
8	Maze-Solving Algorithms	45
9	LMS Implementation Challenge	66
Total		**300**

> **Interview notice:** Questions will be drawn directly from every point made in your submitted deliverables. Be prepared to explain and defend every figure, decision and line of code.

Task 1 — Basic Coding *(15 pts)*

Implement solutions for the following two problems in **C, C++ or Python**:

1. **Line-position estimation:** Given five calibrated, non-negative sensor readings at positions `[-2, -1, 0, 1, 2]`, compute the weighted centroid to estimate line position. Handle the all-zero case explicitly.
2. **Motor-command limiting:** Given signed left/right motor commands and a maximum permitted magnitude, clamp both values while preserving sign.

Deliverables: Runnable source code with at least one example input/output per function, and a brief explanation of logic.

Task 2 — Datasheet Literacy *(18 pts)*

Select **one component** from your BOM (Task 4) — preferably an active component such as the motor driver IC — and answer:

1. Identify two or three datasheet parameters that directly constrain your circuit design (e.g., max current, voltage range). Explain how each influenced a decision in your BOM or wiring.
2. Identify one condition or rating that, if violated, would damage the component. Explain how your design avoids it.

Deliverables: A structured written response referencing specific specs from the datasheet or supplier page (e.g., robu, DFRobot). Page numbers not required.

Task 3 — PID Control *(24 pts)*

Explain how PID control applies to line following:

- The role of the P, I and D terms individually.
- Which configuration (P, PD or full PID) you would select for this robot and why.
- How sensor readings are converted into an error value and then into left/right motor commands.
- How you would respond to oscillation or a lost-line condition.

Deliverables: A written explanation and pseudocode or C++ code sketch. May be incorporated into Task 9.

Task 4 — BOM Preparation *(27 pts)*

Prepare a Bill of Materials for an LMS / Meshmerize robot within **₹10,000**.

Include the controller, sensors, motors, motor driver, battery, chassis, wheels, wiring and connectors. Include shipping and contingency.

For each component, provide:

- Exact model, quantity, unit price and supplier link.
- Key specifications and rationale for selection.

For **one** major component choice (e.g., motor or motor driver), briefly justify why you picked it over at least one alternative.

Deliverables: BOM table and a rough power-block diagram (hand-drawn and photographed is fine).

Task 5 — Power Budget & Battery Selection *(30 pts)*

Produce a power budget for the system in your BOM:

1. **Current draw estimation:** Identify the idle and peak (stall) current draw of every powered component and sum to a worst-case total.
2. **Runtime calculation:** Given your battery's capacity (mAh) and nominal voltage, calculate estimated runtime under typical load.
3. **Cell chemistry justification:** Compare two battery chemistries (e.g., LiPo vs. Li-ion). Justify your choice on energy density, C-rating, weight and cost.

Deliverables: A power budget table, runtime calculation with stated assumptions, and a short chemistry comparison.

Task 6 — Electronics & Hardware *(36 pts)*

Produce a labelled wiring diagram connecting the controller, sensor array, motor driver, motors and power system. Cover:

- Supply voltages, common-ground topology and the functional role of the motor driver.
- A troubleshooting analysis of **one fault** of your choice (e.g., a motor not running, incorrect sensor readings, or the controller resetting when motors engage): state probable root causes, the first diagnostic check you would perform, and how you would confirm the fix.

Deliverables: Wiring diagram and one troubleshooting analysis. Hand-drawn diagrams are acceptable. Task 4 diagrams may be reused.

Task 7 — Wireless Sensor Diagnostics *(39 pts)*

> Assuming the maze surface has small bumps. As the robot runs, one or more IR sensors produce intermittent readings. Identify the faulty sensor(s) in real time, without stopping the robot or touching the hardware.

1. **Protocol selection:** State which wireless protocol available on the ESP32 (Wi-Fi, Classic Bluetooth or BLE) you would use for this diagnostic scenario and justify your choice (latency, power, ease of implementation).
2. **System architecture:** Describe the data flow from sensor read on the ESP32 to visualisation or logging on a phone or PC. A block diagram or a clearly labelled stage-by-stage description is sufficient.

Deliverables: A written protocol justification and a block diagram or structured architecture description. No code required.

Task 8 — Maze-Solving Algorithms *(45 pts)*

Compare **DFS** with a simple **wall-preference rule** (left/right-hand rule). State which you would use for an unknown maze and why.

Using a small example maze, demonstrate:

- How junctions and paths map to nodes and edges.
- How your chosen method explores a route through the maze.
- Distinguish between **exploratory traversal** (unknown maze) and **optimal-route replay** (known map) — explain briefly why the replay phase can use a simpler strategy.

Deliverables: One annotated maze diagram, a brief worked traversal example, and pseudocode for your chosen approach.

Task 9 — LMS Implementation Challenge *(66 pts)*

Using **Appendix A**, implement an ESP32 Arduino C++ program that follows a line with PID control.

The program must:

1. Estimate line position from the IR sensor array using a weighted centroid.
2. Compute a PID error and apply differential motor speed correction.
3. Handle a lost-line condition with a safe stop or a simple recovery.

Validate by running the code against sample sensor inputs (hard-coded arrays are fine). Physical robot testing is not required but will be noted positively.

Deliverables: Commented source code and brief notes on your PID tuning choices.

> Note that we may prepare the hardware and maze for you and ask you to test your code on it.

Optional Extensions

- Maze exploration followed by optimal-route replay.
- Weighted routing or turn-cost modelling.
- Encoder-based closed-loop speed control.
- PCB layout.

Final Deliverable

Submit a **single GitHub repository**:

- `BOM/` — BOM table, power-block diagram, power budget and comparisons.
- `docs/` — Notes on control theory, maze analysis, datasheets, troubleshooting, power budget and wireless diagnostics.
- `src/` — Source code and test cases.
- `README.md` — How to run the code, assumptions, limitations and references.

Target **2–4 pages of notes**, excluding BOM, diagrams, tables and code. Do not duplicate material across tasks.

Appendix A — Mock Hardware Specification Sheet

Platform: ESP32-WROOM-32 (Arduino framework, C++)

IR Sensor Array

Sensor	GPIO Pin	Notes
IR_S1 (leftmost)	GPIO 34	ADC1_CH6 — input only
IR_S2	GPIO 35	ADC1_CH7 — input only

Sensor	GPIO Pin	Notes
IR_S3 (centre)	GPIO 32	ADC1_CH4
IR_S4	GPIO 33	ADC1_CH5
IR_S5 (rightmost)	GPIO 25	ADC1_CH8

- Sensor output: Analog voltage (0–3.3 V); higher voltage indicates a darker surface (line detected).
- Use `analogRead()` (12-bit resolution, range 0–4095). Apply min-max calibration at startup.
- Sensor positions mapped to weights: `[-2, -1, 0, 1, 2]`.

Motor Driver — L298N

Signal	GPIO Pin	Function
ENA	GPIO 14	Left motor PWM speed (0–255 via `analogWrite`)
IN1	GPIO 27	Left motor direction A
IN2	GPIO 26	Left motor direction B
ENB	GPIO 13	Right motor PWM speed
IN3	GPIO 12	Right motor direction A
IN4	GPIO 11	Right motor direction B

- Motor supply: 11.1 V (3S LiPo). Logic supply: 3.3 V from ESP32 3V3 pin.

Power Rail Summary

Rail	Source	Consumers
11.1 V	3S LiPo	L298N motor supply; onboard 5 V regulator input
5 V	L298N 5 V output or separate regulator	ESP32 Vin
3.3 V	ESP32 internal regulator	IR sensors, logic signals

Serial Debug

- Initialise with `Serial.begin(115200)` in `setup()`.

- Stream sensor values, computed error and motor commands at each loop iteration for telemetry and debugging.